

## The Program in Action

When you run *rockpaperscissors.py*, the output will look like this:

---

```
Rock, Paper, Scissors, by Al Sweigart al@inventwithpython.com
- Rock beats scissors.
- Paper beats rocks.
- Scissors beats paper.

0 Wins, 0 Losses, 0 Ties
Enter your move: (R)ock (P)aper (S)cissors or (Q)uit
> r
ROCK versus...
1...
2...
3...
SCISSORS
You win!
1 Wins, 0 Losses, 0 Ties
Enter your move: (R)ock (P)aper (S)cissors or (Q)uit
--snip--
```

---

## How It Works

The game logic for Rock Paper Scissors is fairly straightforward, and we implement it here with `if-elif` statements. To add a bit of suspense, lines 45 to 51 count down before revealing the opponent's move, with brief pauses between counts. This gives the player a period in which their excitement builds about the results of the game. Without this pause, the results would appear as soon as the player entered their move—a bit anticlimactic. It doesn't take a lot of code to improve the user experience for the player.

---

```
1. """Rock, Paper, Scissors, by Al Sweigart al@inventwithpython.com
2. The classic hand game of luck.
3. View this code at https://nostarch.com/big-book-small-python-projects
4. Tags: short, game"""
5.
6. import random, time, sys
7.
8. print('''Rock, Paper, Scissors, by Al Sweigart al@inventwithpython.com
9. - Rock beats scissors.
10. - Paper beats rocks.
11. - Scissors beats paper.
12. ''')
13.
14. # These variables keep track of the number of wins, losses, and ties.
15. wins = 0
16. losses = 0
17. ties = 0
18.
19. while True: # Main game loop.
20.     while True: # Keep asking until player enters R, P, S, or Q.
21.         print('{} Wins, {} Losses, {} Ties'.format(wins, losses, ties))
```